Technische Systemarchitektur für das

Nexus Object-Driven Edge Protocol (Nexus-ODEP).

Vorwort & Systemdefinition

Vorwort des Entwicklers

Die fortschreitende Konvergenz von Informationstechnik (IT) und operativer Technologie (OT) verlangt nach neuen, radikal dezentralen Architekturmustern. Traditionelle Protokolle der Automations- und Veranstaltungstechnik stoßen bei steigender Komplexität, dynamischen Echtzeitanforderungen und unzuverlässigen Netzwerkschichten an ihre strukturellen Grenzen. Das hier spezifizierte Framework bricht mit monolithischen Steuerungsstrukturen und verlagert die Intelligenz, Raumgeometrie und Konfliktlösung direkt an die logische Netzwerkkante (Edge).

Dieses Dokument dient als technologisches Fundament, Architektur-Leitfaden und Implementierungsspezifikation für das *Nexus Object-Driven Edge Protocol (Nexus-ODEP)*. Es sichert die Unabhängigkeit von proprietären Ökosystemen und ebnet den Weg für eine vollständig plattform-, payload- und herstellerunabhängige Echtzeit-Automation über standardisierte IP-Infrastrukturen.

Abstract (Deutsch)

Das **Nexus Object-Driven Edge Protocol (Nexus-ODEP)** ist ein universelles, quelloffenes Meta-Framework und Steuerungsprotokoll für eingebettete Systeme, das Internet der Dinge (IoT), Industrie 4.0 und die professionelle Veranstaltungstechnik. Nexus-ODEP bricht radikal mit traditionellen, zentralisierten Streaming-Architekturen und Cloud-Abhängigkeiten. Durch den konsequenten Einsatz von Edge Computing verlagert das Protokoll die administrative Logik, zeitbasierte Ressourcenplanung und gerätebezogene Konfliktlösung direkt auf dezentrale Netzknoten (Peer-to-Peer).

Nexus-ODEP ist vollständig payload-agnostisch: Es diktiert keine branchenspezifischen Anwendungsbefehle, sondern liefert eine robuste, zeitsynchronisierte strukturelle Hülle für deren konfliktfreie Ausführung. Basierend auf einer hardwarenahen, kalenderbasierten Ressourcen-Engine, die über hochpräzise Zeitprotokolle (IEEE 1588 PTP oder NTP) synchronisiert wird, ermöglicht Nexus-ODEP echten Multi-Controller-Betrieb über TCP/IP. Knoten-Ressourcen können entweder exklusiv reserviert oder über Prioritätswarteschlangen im Shared-Mode genutzt werden. Über virtuelle Geräte (Virtual Devices) agieren Edge-Knoten autonom als Proxy-Server, um untergeordnete Legacy- oder serielle Bussysteme einzubinden. Für unzuverlässige Netzwerkschichten verfügt Nexus-ODEP über eine native Buffer-Ahead-Engine, die Netzwerk-Jitter und Paketverlust durch zeitentkoppelte Befehlspufferung vollständig eliminiert.

Architektonisch umgeht Nexus-ODEP universumsbasierte Verteilungsprinzipien zentraler Master-Datenbanken, indem es systematisch auf abstrakte Virtualisierung und lokalisierte Intelligenz auf der Node-CPU setzt.

Abstract (English)

The **Nexus Object-Driven Edge Protocol (Nexus-ODEP)** is a universal, open-source meta-framework and control protocol designed for embedded systems, the Internet of Things (IoT), Industry 4.0, and professional entertainment technology. Nexus-ODEP fundamentally dismantles traditional centralized streaming architectures and cloud dependencies. By utilizing edge computing, the protocol shifts administrative logic, time-based resource allocation, and device-level conflict resolution directly to the decentralized nodes (peer-to-peer).

Nexus-ODEP is entirely payload-agnostic: it does not dictate sector-specific application commands, but rather provides the robust, time-synchronized structural envelope for their conflict-free execution. Operating on a hardware-level, calendar-based resource allocation engine synchronized via high-precision time protocols (IEEE 1588 PTP or NTP), Nexus-ODEP enables true multi-controller operation over TCP/IP. Node resources (e.g., drive axes, sensors, actuators, or color engines) can be reserved either exclusively or in shared mode utilizing priority queues. Through Virtual Devices (VD), high-performance edge nodes autonomously act as proxy servers to cascade subordinate legacy or serial bus devices. For unreliable network layers (cellular, WAN, internet), Nexus-ODEP features a native Buffer-Ahead Engine that completely mitigates network jitter and packet loss through time-decoupled command pre-buffering.

Architecturally, Nexus-ODEP bypasses universe-based distribution principles of centralized master control databases (protected by third-party proprietary frameworks) by systematically leveraging abstract virtualization and localized intelligence on the node's CPU.

Technische Systemarchitektur: Nexus-ODEP (Teil 1)

1. System-Übersicht & Philosophie

Das **Nexus Object-Driven Edge Protocol (Nexus-ODEP)** ist ein universelles, quelloffenes Meta-Framework und Steuerungsprotokoll für eingebettete Systeme, das Internet der Dinge (IoT), Industrie 4.0 und die professionelle Veranstaltungstechnik.

1.1 Bruch mit traditionellen Architekturen

Klassische Steuerungssysteme basieren auf starren, zentralisierten Streaming-Architekturen (z. B. Art-Net/sACN in der Lichttechnik oder zyklische Master-Slave-Abfragen in der Industrie). Diese Systeme senden permanent gigantische Datenströme (Pixeldaten oder Register-Inhalte) im Kreis. Fällt die Zentrale oder das Netzwerk aus, steht das Gesamtsystem still.

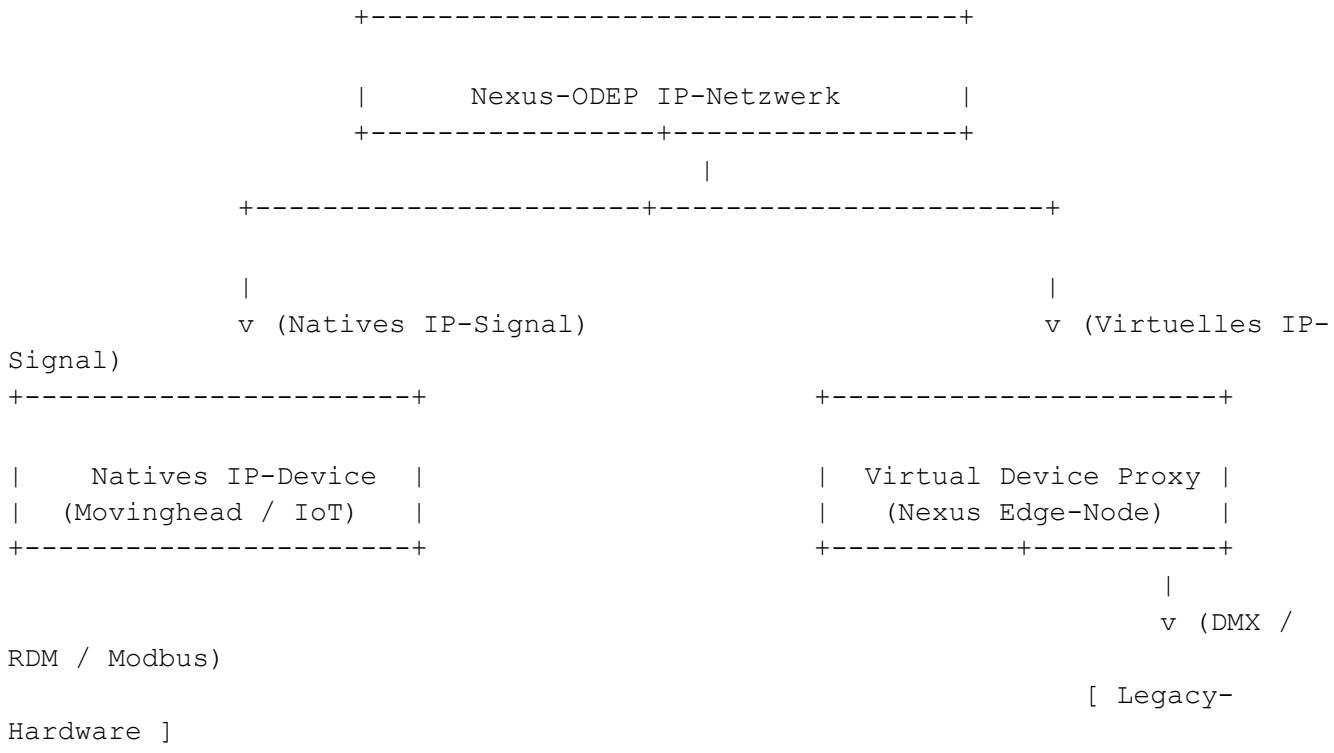
Nexus-ODEP demontiert dieses Prinzip radikal:

- **Dezentralisierung (Edge Computing):** Die administrative Logik, zeitbasierte Ressourcenplanung und Konfliktlösung werden direkt auf die dezentralen Endknoten (Peers) verlagert.

- **Zustandsbasiert statt Streaming:** Es werden keine permanenten Datenströme gesendet. Knoten kommunizieren über mathematische Befehle, Ziel-Zustände (States) und direkte Ereignisse (Events).
- **Payload-Agnostisch:** Das Protokoll diktiert keine branchenspezifischen Anwendungsbefehle, sondern liefert die zeit- und raumsynchronisierte Struktur für deren konfliktfreie Ausführung.
- **Lokale Intelligenz:** Durch Virtualisierung auf Node-Ebene agieren Edge-Knoten als autonome Proxys für ältere Legacy-Geräte.
-

2. Das universelle IP-to-Device Adressierungsmodell

Die fundamentale Säule von Nexus-ODEP ist die **vollständige Auflösung der Grenze zwischen IT-Netzwerken und Hardware-Feldbussen**. Das Internet Protocol (IP) wird als universelles Transferprotokoll direkt bis auf die Geräteebene definiert.



2.1 Native IP-Endpunkte

Verfügt ein modernes Gerät (z. B. ein neuer Movinghead, ein smarter Sensor) über eine eigene Netzwerkschnittstelle (Ethernet, WLAN, Thread), läuft der Nexus-ODEP-Core direkt auf dessen eigener CPU. Jedes Gerät besitzt eine eigene IP-Adresse und kommuniziert im Peer-to-Peer-Verfahren direkt mit anderen Geräten im Subnetz.

2.2 Virtual Device Proxy (VD) für Legacy-Hardware

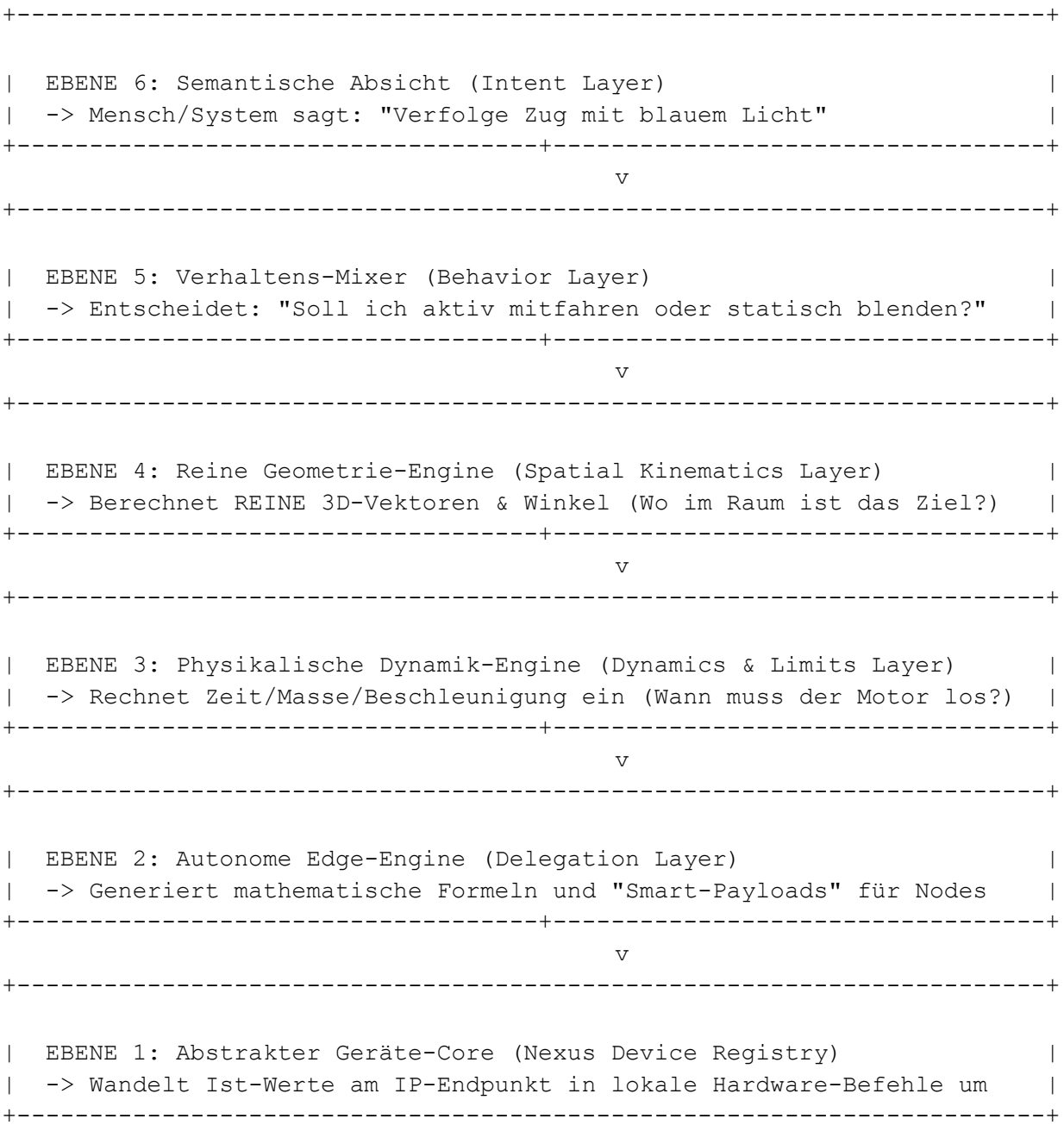
Ältere, serielle Hardware (z. B. DMX/RDM-Scheinwerfer oder Modbus-RTU-Sensoren), die keine eigene IP-Fähigkeit besitzt, wird über einen Nexus-Edge-Node virtualisiert:

- **Virtuelle IPs:** Der Edge-Node weist jedem angeschlossenen "dummen" Gerät eine eigene, virtuelle IP-Adresse zu (oder mappt sie über dedizierte Netzwerk-Ports).
- **Lokaler Protokoll-Tunnel:** Für das übergeordnete Nexus-Netzwerk verhält sich die Lampe wie ein moderner IP-Teilnehmer. Der Edge-Node fängt die IP-Pakete ab, übersetzt sie in

Mikrosekunden in das spezifische Bus-Signal (z. B. RDM-PIDs) und bedient das physische Kabel.

3. Die 6-Schichten-Systemarchitektur

Um eine vollständige Trennung von Absicht, mathematischem Raum, physikalischer Zeit und Hardware-Signalen zu garantieren, ist der Nexus-ODEP-Stack in **sechs streng isolierte Schichten** unterteilt. Jede Ebene kommuniziert ausschließlich mit der Schicht direkt darunter.



3.1 Detaillierte Schichtenbeschreibung

EBENE 6: Semantische Absicht (Intent Layer)

Diese Ebene nimmt die abstrakte, menschliche oder systemseitige Absicht entgegen (z. B. per API, Makro oder Benutzeroberfläche). Sie arbeitet rein namensorientiert und löst Begriffe wie „Zug“ oder „Schlagzeug“ in eindeutige Objekt-IDs auflösen.

- *Beispiel:* "Beleuchte Objekt_Zug auf Bühne_Haupt."

EBENE 5: Verhaltens-Mixer (Behavior Layer)

Der Regisseur des Systems. Diese Schicht entscheidet über die logische Methodik der Umsetzung. Sie gleicht den Wunsch aus Ebene 6 mit den physikalischen Rückmeldungen der unteren Schichten ab.

- *Beispiel:* Kann ein Movinghead dem Zug physikalisch nicht schnell genug folgen, verwirft Ebene 5 für dieses Gerät das Verhalten "Tracking" und aktiviert automatisch das Verhalten "Statisches Crossfade" (Übergabe an die nächste feste Lampe).

EBENE 4: Reine Geometrie-Engine (Spatial Kinematics Layer)

Hier wird die reine, statische 3D-Trigonometrie des Raumes berechnet (unter Nutzung von Matrizen-Bibliotheken wie *Eigen*). Sie kennt die Raumkoordinaten (X, Y, Z) des Ziels sowie die Positionen und Ausrichtungen der Geräte.

- *Beispiel:* Berechnet aus der Position des Zuges den exakten statischen Ziel-Winkel (Pan/Tilt) für Scheinwerfer 1, ohne zu wissen, wie schnell dieser fahren kann.

EBENE 3: Physikalische Dynamik-Engine (Dynamics & Limits Layer)

Diese Ebene bringt die **Zeitkomponente** und die physikalische Masse ins Spiel (unter Nutzung von Robotik-Bibliotheken wie *OMPL/MoveIt2*). Sie liest die mechanischen Limits (Beschleunigung, Maximalgeschwindigkeit) der Geräte aus der Geräte-Datenbank.

- *Beispiel:* Berechnet die exakte Geschwindigkeitsrampe für die Motoren und bestimmt den **Vorausfahr-Zeitpunkt (Move-in-Black)**: *"Weil der Zug in 1,2 Sekunden dort ist, der Motor aber Beschleunigung braucht, muss die Bewegung lautlos vorab bei T minus 0,8 Sekunden starten."*

EBENE 2: Autonome Edge-Engine (Delegation Layer)

Die Schicht der Radikalen Delegation. Anstatt kontinuierlich einzelne Steuerdaten (DMX-Bytes) zu senden, komprimiert diese Schicht die Ergebnisse der Ebenen 3–5 in eine „**Smart-Payload“** (eine mathematische Formel).

- *Beispiel:* Sie generiert einen einmaligen Befehl: *"Starte Regenbogen-Welle mit 0.5Hz von links nach rechts zum exakten PTP-Zeitstempel X"*.

EBENE 1: Abstrakter Geräte-Core (Nexus Device Registry)

Die Schnittstelle zur physischen Realität direkt am IP-Endpunkt. Sie hält den Digitalen Zwilling (Eigenschaften wie `lighting.intensity`) im Speicher. Sie nimmt die mathematischen Aufträge von Ebene 2 entgegen und übersetzt sie direkt auf der Geräte-CPU (oder dem Edge-Proxy) in die realen Signale (z. B. Ansteuerung der LED-Treiber, Schrittmotoren oder die lokale Ausgabe über RDM, MQTT oder Modbus).

4. Parallele Datenpipelines & Radikale Edge-Delegation

Um die Netzwerkbandbreite drastisch zu entlasten und maximale Echtzeitfähigkeit zu garantieren, teilt Nexus-ODEP die Datenverarbeitung ab Ebene 5 in zwei vollkommen autarke, parallel laufende Pipelines auf.

4.1 Die zwei parallelen Pipelines

- **Die Event-Pipeline (Diskrete Zustände):** Zuständig für statische Cues, Schaltbefehle oder reine Parameter-Änderungen (z. B. „Schalte Licht im Bahnhof ein“). Sie arbeitet rein ereignisgesteuert. **Hier fließt kein permanenter Datenstrom über das Netzwerk.** Ein Event wird einmalig übertragen und ändert den Zustand im Zielknoten.
- **Die Stream/Vektor-Pipeline (Kontinuierliche Bewegung):** Zuständig für mathematische Verfolgungen (Tracking) und Kurvenberechnungen (z. B. „Folge dem Zug“). Sie berechnet im Voraus 3D-Trajektorien (Kurven-Splines) und reicht diese an die untergeordneten Schichten weiter.

4.2 Funktionsweise der Radikalen Edge-Delegation

Nexus-ODEP eliminiert das zentrale Streamen von Pixel- und Schrittmotordaten. Stattdessen sendet die Engine ausschließlich **mathematische Aufträge (Smart Payloads)** über das IP-Netzwerk an die Geräte-Endpunkte:

json

```
{
  "command": "START_GENERATIVE_EFFECT",
  "effect_type": "RAINBOW_WAVE",
  "parameters": {
    "speed": "0.5Hz",
    "direction": "left-to-right",
    "phase_offset": 45.0
  },
  "start_time_ptp": 1711234567890123
}
```

Verwende Code mit Vorsicht.

- **Verarbeitung am nativen IP-Node:** Der Mikrocontroller direkt im Gerät (z. B. ein ESP32) empfängt dieses winzige JSON *einmalig*. Seine interne CPU berechnet die Sinuswelle für die LED-Treiber vollkommen autark und absolut jitterfrei im Mikrosekunden-Takt. Das Netzwerk ist danach sofort wieder frei.
- **Netzwerkausfall-Resistenz:** Reißt die Netzwerkverbindung während einer Zugverfolgung oder eines Farbeffekts für mehrere Sekunden ab, läuft der Effekt auf dem Gerät perfekt synchronisiert weiter, da das Endgerät seine mathematische Kurve und die exakte PTP-Systemzeit kennt.

5. Bidirektionales Peer-to-Peer (P2P) Routing & Makro-Laufzeitumgebung

Nexus-ODEP-Nodes verhalten sich wie ein dezentralisiertes IP-Netzwerk für Automations-Zustände. Die Ebenen 1 und 2 arbeiten **vollständig bidirektional**. Sie verarbeiten Daten

nicht nur von oben nach unten (Top-Down), sondern werfen Hardware-Events auch von unten nach oben (Bottom-Up) in das abstrakte Modell.

5.1 Das Prinzip der Direktverknüpfung (Direct Binding)

Wenn ein physischer Sensor (z. B. eine Lichtschranke am Gleis) ein Gerät (z. B. einen Dimmer-Kanal im Zug) abschalten soll, läuft dieser Vorgang komplett auf der Edge-Ebene ab – ohne Umweg über eine übergeordnete Steuerung:

1. Der Sensor registriert eine Flankenänderung.
2. Ebene 1 abstrahiert das Signal in eine universelle Eigenschaft (z. B. `sensor.occupancy = true`).
3. Ebene 2 prüft die lokale Routing-Tabelle und sieht ein aktives Binding: *„Wenn `sensor.occupancy == true`, sende ein P2P-Paket direkt an die IP von `DMX_Node_4` und setze `lighting.intensity = 0`“*.
4. Der gesamte Vorgang findet innerhalb des lokalen Subnetzes statt und benötigt **unter 0,5 Millisekunden**.

5.2 Lokale Makros und Ortsunabhängigkeit (Location Transparency)

Auf den Nodes können komplexe logische Makros (Sequenzen) lokal hinterlegt und ausgeführt werden. Diese Makros greifen über eine abstrakte Netzwerk-Schnittstelle auf Ressourcen zu.

Dem Makro ist es vollkommen gleichgültig, ob ein Sensor direkt am selben Node verkabelt ist oder per MQTT auf einem anderen Node am anderen Ende der Halle liegt. Der lokale **Nexus Routing Core** (der "Hub") entscheidet asynchron:

- **Lokaler Treffer:** Die Ressource liegt am eigenen Node → Der Wert wird im RAM in Nanosekunden ausgelesen.
- **Externer Treffer:** Die Ressource liegt auf einem entfernten Node → Der Core verpackt die Anfrage in ein minimales, binäres UDP-Paket und schickt es per P2P an den Ziel-Node. Dieser antwortet auf exakt demselben Weg (Request-Response-Rückkanal).

6. Validierter Open-Source C++ Tech-Stack

Für die kommerzielle, lizenzrechtlich unbedenkliche Umsetzung des Nexus-ODEP-Frameworks in C++ werden ausschließlich hochperformante Bibliotheken mit freizügigen Lizenzen (MIT, Apache 2.0, BSD, LGPL) empfohlen.

6.1 Infrastruktur- & Echtzeitschicht

- **Betriebssystem-Kern (Echtzeit):** **`Linux mit PREEMPT_RT Patch`**. Ermöglicht deterministisches Thread-Scheduling und verhindert unvorhersehbare Latenzen des Betriebssystems. Zeitkritische Threads werden via POSIX (`SCHED_FIFO`) priorisiert und fest an dedizierte CPU-Kerne gebunden (`pthread_setaffinity_np`).
- **Netzwerk-Kern (Asynchroner TCP/IP-IO):** **`Asio (Standalone)`** oder **`Boost.Asio`** (Boost-Lizenz). Bildet das extrem stabile und performante Fundament für das asynchrone Senden und Empfangen der P2P-IP-Pakete.

- **Jitter-Kompensation & Puffer (Buffer-Ahead):** `MoodyCamel ConcurrentQueue` (Vereinfachte BSD-Lizenz). Ein extrem schneller, lock-free und thread-sicherer Ringpuffer (`readerwriterqueue.h`). Verhindert jegliche Blockierung des zeitkritischen Schedulers durch langsame Netzwerk-Treiber.

6.2 Daten- & Geometrieschicht

- **Offline-Gerätedatenbank (JSON):** `nlohmann/json` (MIT-Lizenz). Zum performanten Einlesen und Parsen der lokalen *Open Fixture Library (OFL)*.
- **Lokales GDTF-Parsing (XML):** `pugixml` oder `TinyXML-2` (MIT/zlib-Lizenz). Leichtgewichtige, schnelle Parser, um GDTF-Gerätespezifikationen direkt offline im Node zu verarbeiten.
- **3D-Vektormathematik (Ebene 4):** `Eigen` (MPL2-Lizenz / kommerziell frei). Die führende C++ Matrix-Bibliothek für die Berechnung der statischen 3D-Trigonometrie des Raumes (Ausrichtung der Moving-Heads auf Ziel-Koordinaten).
- **Trajektorien-Planung & Inverse Kinematik (Ebene 3):** `OMPL (Open Motion Planning Library)` (BSD-Lizenz). Berechnet die zeit- und beschleunigungsoptimierten Flugbahnen für die Bewegung der Geräte-Achsen.

6.3 Protokoll-Adapter (Ebene 1)

- **Bühne / Entertainment:** `Open Lighting Architecture (OLA)` (LGPL-Lizenz für Client-Bibliotheken). Übernimmt die standardkonforme Generierung von Art-Net- und RDM-Paketen für Legacy-Geräte.
- **IoT / Home Automation:** `Eclipse Paho C++` oder `Eclipse Mosquitto` (EPL/EDL-Lizenz). Garantiert die schlanke Anbindung an MQTT-Netzwerke inklusive nativer Unterstützung für Home-Assistant-Discovery.
- **Industrie 4.0:** `libmodbus` (LGPLv2.1-Lizenz). Der weltweite C/C++ Standard für die zuverlässige Kommunikation mit industriellen Modbus-TCP- und RTU-Registern.

7. Unterstützte Industrie- & Geräteprotokolle

Durch den payload-agnostischen Aufbau von Nexus-ODEP übersetzen die Protokoll-Adapter auf Ebene 1 die abstrakten Geräteeigenschaften nativ in die wichtigsten Steuerungsstandards der verschiedenen Branchen:

7.1 Veranstaltungstechnik (Professional Entertainment)

- **Art-Net (v1 - v4):** Ethernet-basiertes Streaming von DMX-Daten zur Ansteuerung von Lichtkomponenten.
- **sACN (ANSI E1.31):** Standardisiertes Multicast-Streaming-Protokoll für sehr große DMX-Datenmengen.
- **RDM / RDMNet (ANSI E1.20 / E1.33):** Bidirektionale DMX-Erweiterung für die Konfiguration (GET/SET) und das Monitoring von Geräten.

7.2 Gebäude- & Heimautomatisierung (Home Automation)

- **Matter:** Der moderne, IP-basierte (Ethernet/WLAN/Thread) Smart-Home-Standard für herstellerübergreifende Interoperabilität.

- **KNX (KNXnet/IP):** Der drahtgebundene Industriestandard für die gehobene, professionelle Gebäudeinstallation.
- **DALI-2 / DALI-3:** Digital Addressable Lighting Interface, spezialisiert auf die architektonische Licht- und Dimmersteuerung im Zweckbau.

7.3 Industriesteuerung (Industry 4.0 & IoT)

- **Modbus (TCP / RTU):** Der robusteste und am weitesten verbreitete Standard für industrielle Registerabfragen bei Sensoren und Antrieben.
- **OPC UA:** Der plattformunabhängige, serviceorientierte Standard für die sichere, maschinenübergreifende M2M-Datenerfassung.
- **MQTT (inkl. Sparkplug B):** Das dominierende, eventbasierte Publish/Subscribe-Protokoll für extrem schlanke IoT-Architekturen.

8. Implementierungs-Leitfaden & Code-Architektur (Checkliste)

Um das Nexus-ODEP-Framework Schritt für Schritt in C++ aufzubauen, wird die Entwicklung in vier aufeinander aufbauende Phasen unterteilt.

Phase 1: Die Daten- & Threadingbasis (Ebene 1 & Kernel)

1. **Speicher-Vorallokierung (Memory Pools):** Implementiere eine feste Objekt-Pipeline für Befehls- und Event-Strukturen, um im laufenden Betrieb jegliche Heap-Allokation (`new`, `malloc`) in den Echtzeit-Threads zu unterbinden.
2. **Thread-Isolation:** Richte den zeitkritischen Core-Scheduler in einem eigenen Thread ein, befehle ihm die höchste POSIX-Priorität (`SCHED_FIFO`) und binde ihn an einen dedizierten CPU-Kern.
3. **Lock-Free Queues:** Binde die `MoodyCamel::ReaderWriterQueue` ein, um die asynchrone Kommunikation zwischen dem Core-Thread und den Netzwerk-Worker-Threads (OLA, Mosquitto) völlig blockierungsfrei zu gestalten.

Phase 2: Die Abstrakte Registry & OFL-Parsing (Ebene 1 & 2)

1. **OFL-Parser:** Schreibe das Lese-Modul mit `nlohmann/json`, das die JSON-Dateien der lokalen *Open Fixture Library* beim Systemstart einliest und eine effiziente Lookup-Map (`std::unordered_map`) basierend auf der RDM-Modell-ID erstellt.
2. **Digitaler Zwilling:** Implementiere die Klasse `AbstractDevice` mit einem generischen Property-Tree (`std::variant`), um Geräte-Zustände hersteller- und protokollneutral im RAM zu verwalten.
3. **Bidirektionaler Event-Bus:** Implementiere die Callbacks, die bei einer Wertänderung von unten (z. B. MQTT-Sensor meldet Zustand) oder von oben (z. B. Makro sendet Befehl) die abstrakte Routing-Tabelle triggern.

Phase 3: Die Raum-, Zeit- & Geometrie-Engine (Ebene 3, 4 & 5)

1. **3D-Koordinaten (Ebene 4):** Binde die Bibliothek `Eigen` ein. Definiere die Transformationen (Matrizen), um die Positionen von Objekten (z. B. `Schlagzeug`) im Raum mit den

mechanischen Nullpunkten der Moving-Heads zu verknüpfen. Implementiere die Inverse Kinematik zur Berechnung der Pan/Tilt-Winkel.
2. **Trajektorien-Planung (Ebene 3):** Integriere `OMPL`, um Kurvenverläufe (Splines) über der Zeitachse zu berechnen und die maximalen Beschleunigungslimits der Motoren mathematisch zu berücksichtigen.
3. **Vorausfahr-Scheduler:** Implementiere die Berechnungslogik für das *Move-in-Black*-Timing, damit Scheinwerferfahrten exakt um ihre mechanisch benötigte Trägheitszeit vor dem eigentlichen Licht-Event gestartet werden.

Phase 4: Die Makro-Laufzeitumgebung & P2P-IP-Routing (Ebene 2 & Netz)

1. **Location Transparency Engine:** Erweitere das Netzwerk-Interface so, dass bei jedem Gerätezugriff geprüft wird, ob das Ziel lokal angeschlossen ist oder auf einem entfernten Peer liegt.
2. **Asynchroner RPC-Kanal:** Nutze `Boost.Asio`, um bei externen Gerätezugriffen ein minimales, binäres UDP-Paket an die Ziel-IP zu schießen und die Antwort über das Request-Response-Pattern an das lokale Makro zurückzureichen.
3. **Smart Payloads:** Implementiere die Generierung der kompakten mathematischen Befehle, damit die Datenströme lokal auf der Edge-Ebene ausgeführt werden, statt das IP-Netzwerk zyklisch zu belasten.

9. Vollständige Protokoll- & Bibliotheks-Matrix

Referenz und schnelle Übersicht für alle im System integrierten Protokollstacks und C++ Bibliotheken, strukturiert nach ihrer jeweiligen technologischen Domäne.

9.1 Software-Bibliotheken (C++ Frameworks)

- **`linuxptp`** (GPLv2 – als separater System-Daemon kommerziell unbedenklich): Hardwaresnahe IEEE 1588 PTPv2 Zeitsynchronisation.
- **`Asio (Standalone)`** / **`Boost.Asio`** (Boost-Lizenz): Asynchrone, nicht-blockierende TCP/IP Netzwerk-Kommunikation.
- **`MoodyCamel ConcurrentQueue`** (Vereinfachte BSD-Lizenz): Lock-free, thread-sichere Ringpuffer für Jitter-Kompensation und Inter-Thread-Kommunikation.
- **`nlohmann/json`** (MIT-Lizenz): Extrem performanter JSON-Parser für die lokale Geräte-Konfiguration und den OFL-Import.
- **`pugixml`** / **`TinyXML-2`** (MIT/zlib-Lizenz): Leichtgewichtige XML-Parser zum Einlesen lokaler GDTF-Gerätebeschreibungen.
- **`Eigen`** (MPL2-Lizenz): Hochleistungs-Matrixbibliothek für die statische 3D-Geometrie und Orientierungsberechnung im Raum.
- **`OMPL (Open Motion Planning Library)`** (BSD-Lizenz): Robotik-Engine für zeit- und beschleunigungsoptimierte 3D-Trajektorienplanung.
- **`Open Lighting Architecture (OLA)`** (LGPL für Client-Bibliotheken): C++ Framework für standardkonformes Art-Net- und RDM-Handling.
- **`Eclipse Paho C++`** / **`Eclipse Mosquitto (libmosquitto)`** (EPL/EDL-Lizenz): Performante MQTT-Clientbibliotheken für die IoT-Integration.

- `libmodbus` (LGPLv2.1-Lizenz): Industrieller C/C++ Protokollstack für Modbus-TCP und Modbus-RTU.

9.2 Kommunikations- & Geräteprotokolle (Netzwerkschicht)

🎭 Unterhaltungstechnologie & Bühne

- **Art-Net (v1 - v4):** IP-basiertes DMX-Streaming für pixelgenaue Lichtsteuerung (Legacy-Output).
- **sACN (ANSI E1.31):** Multicast-Streaming-Architektur für massiven DMX-Datendurchsatz.
- **RDM / RDMNet (ANSI E1.20 / E1.33):** Bidirektionale Konfiguration, Abfrage (GET/SET) und Gerätemanagement über IP.
- **GDTF / MVR (DIN SPEC 15800):** Offene Datenaustausch-Formate für physikalische Geräteeigenschaften (OFL-Basis) und 3D-Szenen.

🏠 Gebäude- & Heimautomatisierung

- **Matter:** IP-basierter Konnektivitätsstandard (WLAN, Thread, Ethernet) für herstellerübergreifende Interoperabilität.
- **KNX (KNXnet/IP):** Dezentrales Bus-System für die professionelle Gebäude- und Zweckbau-Installation.
- **DALI-2 / DALI-3:** Spezialisiertes digitales Lichtbus-Protokoll für Architekturbeleuchtung und EVGs.

🏭 Industriesteuerung & IoT (Industry 4.0)

- **Modbus (TCP / RTU):** Registerbasiertes Standardprotokoll für Sensoren, Aktoren und Frequenzumrichter.
- **OPC UA (IEC 62541):** Plattformunabhängiges, sicheres M2M-Kommunikationsmodell für semantische Industriedaten.
- **MQTT (inkl. Sparkplug B):** Ereignisgesteuertes Publish/Subscribe-Protokoll für extrem bandbreitenschonende IoT-Kommunikation.